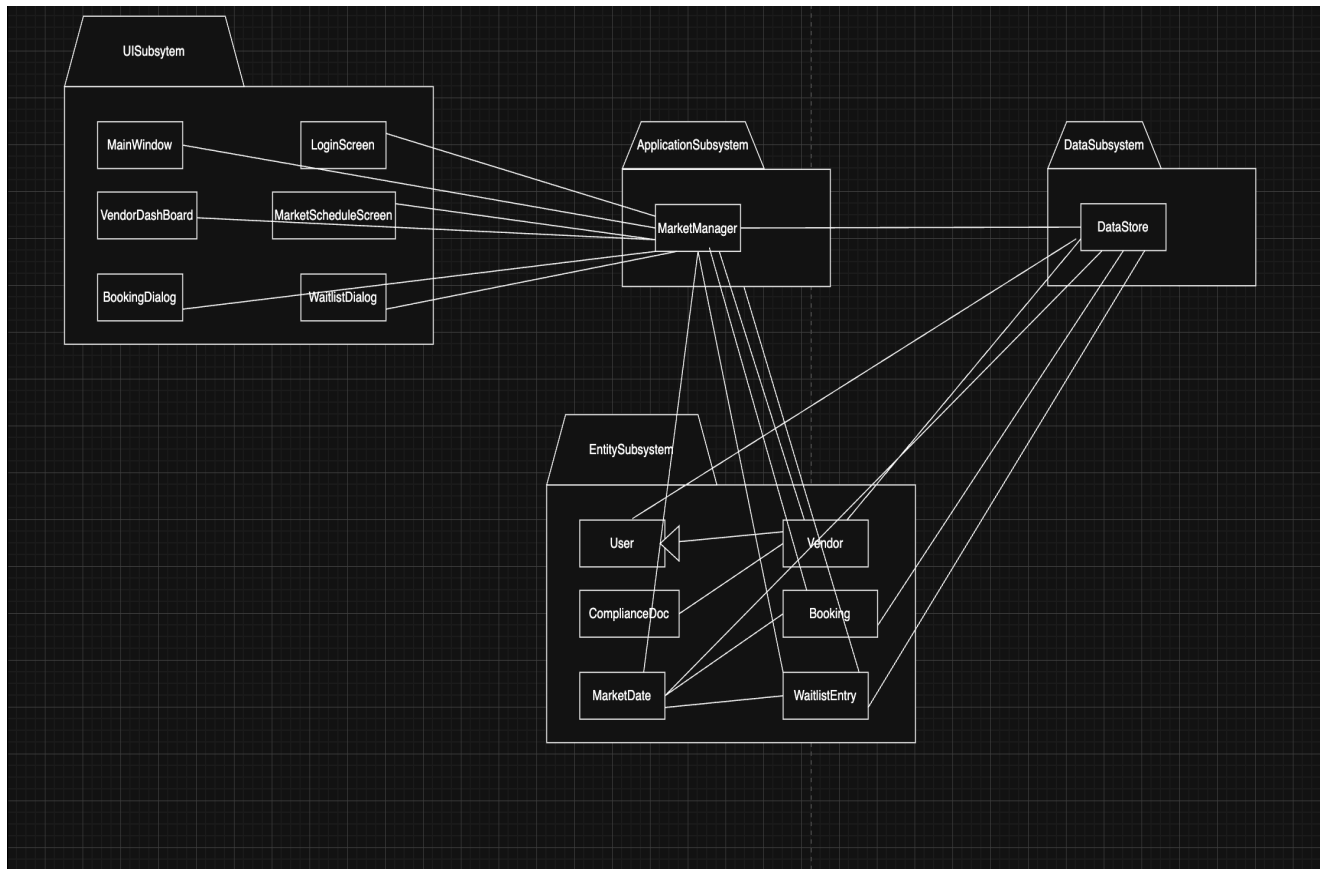


COMP 3004 Winter 2026
HintonMarket Deliverable 1

Title: HintonMarket System Decomposition and Design Explanation
Date: March 12, 2026
Team Number: #60

Team Members:
Harleen Kaur. - 101291778
Olivia Petherick - 101309045
Amber Hashwani - 101307814

Figure 1: HintonMarket System Decomposition Showing Subsystem Organization and Dependencies



Subsystem Descriptions

UISubsystem — The UISubsystem is responsible for all user-facing interactions in the HintonMarket system. It contains the boundary classes that handle login, vendor dashboard display, market schedule browsing, stall booking, and waitlist management. This subsystem communicates exclusively with MarketManager and has no direct knowledge of data storage or entity logic.

ApplicationSubsystem — The ApplicationSubsystem is responsible for coordinating all use case logic in the HintonMarket system. It contains the single control class MarketManager, which acts as the sole mediator between the UISubsystem and the DataSubsystem, handling vendor login, stall booking, booking cancellation, waitlist operations, and compliance verification.

EntitySubsystem — The EntitySubsystem is responsible for representing the core domain objects of the HintonMarket system. It contains all entity classes including users, vendors,

compliance documents, market dates, bookings, and waitlist entries. These classes encapsulate domain data and enforce domain-level rules.

DataSubsystem — The DataSubsystem is responsible for managing all in-memory data collections in the HintonMarket system. It contains DataStore, which seeds initial system data, manages all object collections, and enforces data-level business rules such as booking availability and waitlist processing.

HintonMarket System Design: Coupling and Cohesion

Overview

When we were designing HintonMarket, our main goal was to make sure each part of the system had a clear job and that the different parts didn't depend on each other more than they needed to. We split the system into four subsystems: **UISubsystem**, **ApplicationSubsystem**, **EntitySubsystem**, and **DataSubsystem**. The decisions we made about what goes where were all driven by trying to keep things clean and maintainable.

Minimizing Coupling

One of the first things we decided early on was that the UI should never talk directly to the data layer. In practice this means every screen — LoginScreen, VendorDashboard, MarketScheduleScreen, BookingDialog, WaitlistDialog, and MainWindow — only ever calls methods on MarketManager. None of them know DataStore even exists. We felt this was important because if we ever need to change how data is stored (say, adding a database in a later deliverable), we shouldn't have to touch any of the UI code at all.

MarketManager sitting in the middle as the only bridge between the UI and the data layer was a deliberate choice. It means there's one place where use case logic lives, and the UI just tells it what the user wants to do. DataStore on the other hand only worries about managing the in-memory collections — it doesn't know or care what screen triggered a request.

We also made sure entity classes like Vendor, Booking, and WaitlistEntry don't reference anything outside their own subsystem. They don't know about the UI, they don't know about DataStore, they just represent the domain objects. This keeps them simple and reusable.

Maximizing Cohesion

Each subsystem ended up with a focused set of responsibilities which we think is what kept the code manageable.

The UISubsystem is purely about showing things to the user and reacting to what they do. For example VendorDashboard just takes data it's given and displays it — it doesn't calculate whether a vendor is compliant or query bookings itself. That's not its job.

MarketManager handles all the use case coordination — login, booking, cancellation, waitlist operations, compliance checks. Everything that represents "what the system does" lives there. It delegates the actual data work to DataStore and trusts entity classes to enforce their own domain rules.

Speaking of which, we put domain logic where it naturally belongs. ComplianceDoc knows how to check if it's still valid for the season. Vendor knows how to figure out which documents it's missing. It made more sense to keep that logic inside those classes rather than having MarketManager or DataStore do it.

DataStore's only job is managing the in-memory collections and making sure data-level rules are followed — things like preventing a vendor from booking two stalls at once, or automatically notifying the next vendor on the waitlist when a stall opens.

Summary

The overall structure came down to one simple rule we kept coming back to: each class should have one reason to change. If the UI needs updating, we shouldn't need to touch the data layer. If the storage strategy changes, the UI shouldn't care. We think the four-subsystem breakdown reflects that pretty well and kept the codebase much easier to reason about as we were building it.